

Dynamic model of F450 Quadcopter in Simulink

Key Words : GNC, UAV, Quadcopter, Simulation, Motor Dynamics, Aerodynamics, Body-kinematics, velocity constant, 1st order system, NED, ECI, Max flight time, transport Theorem

서 론

이번 유도제어시스템설계 과목에서는 강의 내용과 실습을 바탕으로 Simulink를 통해 DJI 사의 F450의 동특성을 최종적으로 도출하는 과정, 즉 Dynamic modeling을 진행하였다. 이러한 과정은 최근 무인 항공 시스템(UAS, Unmanned Aerial Systems)의 활용도가 군사 목적을 넘어 민간 분야로도 빠르게 확산하고 있다. 이러한 시스템의 핵심 기술은 유도(Guidance), 항법(Navigation), 제어(Control)를 의미하는 GNC다. 이러한 시스템의 거동을 정확히 예측하고 제어를 설계하기 위한 첫걸음이라 할 수 있다. 이번 보고서에서는 Input을 모터의 속도를 받아 크게 3가지의 Dynamic system의 계산을 거쳐 동특성을 얻는 과정을 단계적으로 설명하고, 달라지는 모터의 속도에 대한 시뮬레이션 결과를 분석하는 방향으로 마무리된다.

본 론

1) Definition of Dynamic systems(1주차)

먼저, F450의 Dynamics systems를 정의하기 전에 이러한 시스템 설계에 대한 정의와 목적 등을 확인해 볼 필요가 있다. 그 의미를 정확히 알고 설계를 진행한다면 더욱 안정하고 구체적인 시스템 엔지니어링이 가능하기 때문이다. 이것이 바로 GNC의 목적과도 같다. GNC가 적용된 UAS의 구성은 아래 다이어그램과 같다.

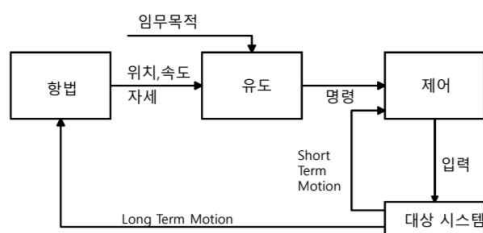


Fig. 1. Diagram of GNC system

먼저, 사용자가 원하는 설정값 (임무목적을 의미함. 예를 들어, 드론의 전진을 위해 모터값을 다르게 조절하는 경우)을 이 시스템에 넣었을 때, 센서에 의해 측정된 제어 대상 시스템인 드론의 현재 위치, 상태, 속도, 각속도 등과 설정값(Command) 간 error를 계산한다. 이러한 error를 guidance에 따라 그 error를 최적화하는 방안에 따라 그 값을 줄이도록 하기 위해 제어기(Control)에서 이를 제어한다. 이렇게 제어값을

Input으로 제어 대상 시스템에 적용하여 설정값에 가까워지도록 반복적으로 과정을 수행하게 된다.

이러한 과정을 현대에서는 고전적 방식이 아닌 정밀해진 임베디드 소프트웨어 및 경량화되고 값싼 하드웨어인 MEMS 센서를 적용함으로써 빠르고 효율이 높은 제어 시스템을 구축할 수 있게 되었다. 앞으로는 더 나아가 AI의 지능형 알고리즘을 드론에 적용해 자율 비행제어가 가능한 드론 플랫폼을 널리 운용하게 될 것이다. 우리는 미래의 과정을 진행하기 위해 F450의 Dynamic modeling을 통해 기초적 단계를 학습하였다. 여기서 Dynamic modeling을 진행하는 중요한 이유 2가지를 인지하고 넘어가야 한다. 먼저, System analysis를 통해 다양한 환경 속 영향을 받는 비행체의 안정성, 조종성, 전력 소모 등의 문제점 등을 해결하는 것이고, Control laws design and simulation을 통해 실패 가능성이 존재하는 테스트를 실제 환경이 아닌 가상으로 구현하여 돈과 시간을 절약할 수 있다는 점에 있어 Dynamic modeling을 진행하게 되는 것이다.

2) Modeling Motor Dynamic system(2주차)

2주차에서는 앞서 언급한 3가지의 Dynamic system에서 Motor Dynamics system을 설계했다. 여기서 Motor Dynamics System이 가지는 물리적 현상과 그에 관한 수학적 모델링을 수행했다. 그리고 기본적인 parameters를 설정하고 결론적으로 F450에 사용되는 모터의 속도를 시간에 따라 어떤 양상을 지니는지 확인하였다.

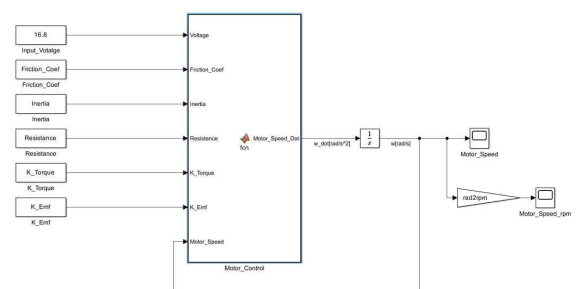


Fig. 2. Simulink diagram of Motor Dynamics

오른쪽에 놓여진 상수 블록들은 위에서부터 단일 모터에 입력되는 전압과 아래는 Motor_Control function에서의 계산 과정에 필요한 파라미터들이며, 이를 받아 w_{dot} 을 도출하여 시간에 따라 적분기를 통해 모터의 속도 w 를 구하는 것이다. 이를 계산하는 과정은

아래의 수식 과정을 통해 설명한다.

Table 1. Motor Dynamics parameters

Symbol	Value	Unit	Description
v_i	16.8	V	입력 전압
b_f	0.0001	Nm/s	마찰계수
J_m	2.8×10^{-5}	kg*m/s ²	관성모멘트
R	0.1170	Ω	저항
K_T	$\frac{60}{2\pi*960}$	N m/A	토크 상수
K_e	$\frac{60}{2\pi*960}$	V s/rad	Emf 상수

수업에서는 모터에서 발생하는 전체적인 토크 τ 를 모터의 회전축에 의한 토크와 회전에 의해 모터 내 볼베어링에서 발생하는 마찰로 생기는 역 토크의 합으로 정의했다.

$$\tau = J_m \frac{dw}{dt} + b_f w \quad [\text{Nm}] \quad (1)$$

여기서 J_m 은 모터의 회전축 기준 모터의 관성모멘트를, b_f 는 마찰계수를 의미한다. 이후에는 마찰계수를 공기역학적 관점으로 확장하겠지만, 현재는 모터의 관점으로만 접근했음을 주의한다. 교안에서는 토크를

$$\tau = K_T i \quad [\text{Nm}] \quad (2)$$

토크 상수와 전류의 곱으로 정의했다. 이를 위 식에 대입하여 시간에 따른 w 의 순간변화율은 다음과 같다,

$$\frac{dw}{dt} = \frac{K_T i - b_f w}{J_m} \quad [\text{rad/s}^2] \quad (3)$$

또한, 모터와 연결되어 있는 전자회로에 키르히호프 제 2법칙인 ‘폐쇄 회로에서 기전력의 합이 전압 강하의 합과 같다’의 원리로 전자회로에 구성된 Armature Inductance, L 을 기계 역학적 관점에서 빠르게 일어나기에 0에 가깝다는 가정을 하였고, 이를 적용해 전자회로에 흐르는 전류 i 에 대한 식을 다음과 같이 정의했다.

$$i = \frac{v_a - K_e w}{R} \quad [\text{A}] \quad (3)$$

여기서 v_a 는 기전력의 전압, K_e 는 Emf 상수이며, R 은 저항을 의미한다. 최종적으로 전기적 손실($R = 0$)을 가정한 상태로 에너지 보존 법칙에 따라 전압이 들어가는 Electrical Power in 에너지와 Mechanical

Power out 에너지의 크기를 같아지므로 Emf 상수와 토크 상수를 같게 둘 수 있게 된다. 이를 통해 구할 수 없던 Emf 상수를 토크 상수를 통해 모터의 속도를 구할 수 있게 되었다. 토크 상수는 모터의 속도 상수의 역수 관계라는 사실을 통해 구할 수 있으며, F450에 사용되는 모터의 속도 상수는 960Kv로 주어져 있다. 이를 식으로 나타내면 다음과 같다.

$$K_T = K_e = \frac{1}{K_v} = \frac{\tau}{i_a} = \frac{60}{2\pi*(960)} \quad [\text{Nm/A}] \quad (4)$$

위 식은 육십분법으로 표현된 식이므로 RPM 단위로 바꾸려면 위의 값에 $2\pi/60$ 을 곱하여 사용한다. 이는 rad2rpm이라는 gain 블록을 통해 파라미터로 변환하였다. 위의 정의된 식을 matlab function에 구현한 과정은 아래와 같다.

```
function Motor_Speed_Dot = fcn(Voltage, Friction_Coeff, Inertia, Resistance, K_Torque, K_Emf, Motor_Speed)
V_Emf = K_Emf * Motor_Speed; %[Voltage]
Current = (Voltage - V_Emf) / Resistance; %[A]
Motor_Speed_Dot = (K_Torque * Current - Friction_Coeff * Motor_Speed)/ Inertia; %[rad/s^2]
end
```

Fig. 3. Motor Dynamics function

앞서 구한 (1)식을 w 에 대한 미분방정식을 라플라스 변환했을 때, 식은 다음과 같다.

$$G(s) = \frac{C(s)}{R(s)} = \frac{k_s}{s(\tau s + 1)} \quad (5)$$

이는 t-domain에서의 상수인 입력전압의 s-domain인 $a \cdot 1/s$ (a 는 상수) 꼴과 Motor dynamic system의 s-domain인 1차시스템의 표준형의 곱으로 이뤄짐을 알 수 있다. 이에 대한 t-domain 함수 꼴은

$$w(t) = k_s v_i (1 - e^{-\frac{t}{\tau}}) \quad (6)$$

로 나타나게 되고, 이를 simulink 내 scope로 결과를 나타내면 아래와 같이 단조 증가 함수의 형태로 나타나게 된다.

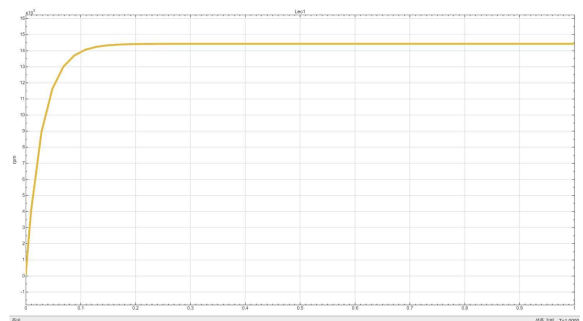


Fig. 4. t-domain Motor speed graph

03) Expansion Motor Dynamic system(3주차)

3주차에서는 앞서 언급했듯이 모터 관점의 마찰계수를 공기역학적 관점에서 확장하여 마찰계수를 적용할 필요가 있다. 그 이유는 모터 자체적으로 발생하는 마찰력보다 모터에 달려있는 프롭에 의해 발생하는 항력에 의한 모멘트가 토크값에 큰 영향을 주기 때문이다. 이를 정의하고자 먼저 프롭이 발생시키는 추력 T와 모멘트 F의 정의를 확인해야 한다.

$$T = \frac{1}{2} \rho C_T A v^2 = \frac{1}{2} \rho C_T A (Rw)^2 = bw^2 \text{ [N]} \quad (7)$$

$$F = \frac{1}{2} R \rho C_D A v^2 = \frac{1}{2} R \rho C_D A (Rw)^2 = dw^2 \text{ [Nm]} \quad (8)$$

공기역학적관점에서, 추력은 그 주위 공기의 밀도와 추력계수, 그리고 유량이 통과하는 단면적과 그때의 유속의 제곱에 비례한다. 여기서 선속도인 유속을 각 속도의 개념으로 'v = rw'를 적용하여 변환한다. 그렇게 했을 때, 각속도 관점에서의 추력계수와 항력계수 각각 b, d는 다음 식을 따르게 된다.

$$b = \frac{1}{2} \rho C_T A R^2 \text{ [N*s}^2\text{]} \quad (9)$$

$$d = \frac{1}{2} \rho C_D A R^3 \text{ [Nm*s}^2\text{]} \quad (10)$$

3주차에서의 목적은 F450의 Hovering 상태를 만들어주는 Motor Dynamic system으로 개선시키는 것이다. 그러기 위해 먼저 드론의 Hovering 상태를 정의해야 한다.

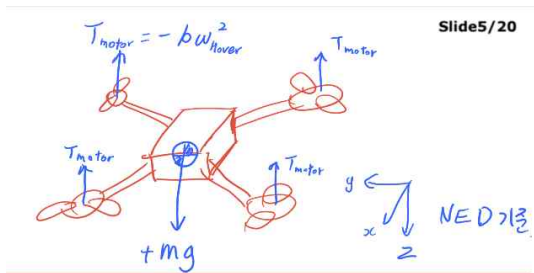


Fig. 5. Hovering state of F450

위의 F450 Quadcopter가 hovering상태로 공중에 가만히 정지해있다고 하자. 그렇다면, 정역학적 관점으로 드론에 장착된 4개의 모터가 발생시키는 추력의 총합이 드론의 중력의 크기와 같아야 하는 힘의 평형이 이뤄져야 한다. (여기서 모멘트적 관점으로는 중력이 드론의 무게 중심에 놓여져 있고, 무게중심을 기준으로 대칭적으로 모터의 추력이 작용하고 있으므로 모멘트 총합은 0이 된다.) 그래서 이를 수식적으로 나타내면 다음의 수식을 만족하게 된다.

$$|4 \vec{T}_{motor}| = |\vec{mg}|, \quad T_{motor} = \frac{mg}{4} = bw^2 \quad (11)$$

$$w_{hover} = \sqrt{\frac{1}{4} mg/b} \text{ [rad/s]} \quad (12)$$

구해진 식에 아래의 파라미터를 통해 호버링 속도를 구한다.

Table 2. Hovering state parameters

Symbol	Value	Unit	Description
b	1.1768×10^{-4}	V	추력계수
m	1.5	kg	F450 질량
g	9.8066	m/s ²	중력가속도

위 파라미터를 대입했을 때 나온 F450의 호버링 속도는 559.11rad/s임을 알 수 있다. 이제는 Input을 모터에 입력되는 전압이 아닌 호버링 속도(또는 사용자가 입력하는 속도)를 넣으면 앞서 알아본 관계식을 통해 전압을 따로 구해 MotorDynamic system을 돌리게 하는 function을 만들었다.

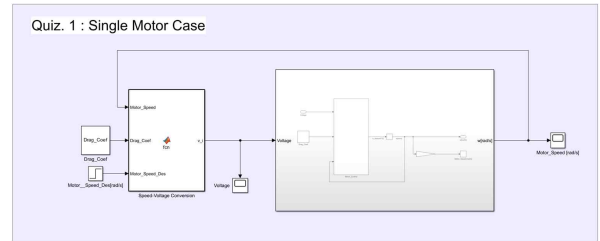


Fig. 6. Applying w_d to v_i conversion

위에서 적용된 Drag_Coeff, $d = 1.8542 \times 10^{-7} \text{ [Nm*s}^2\text{]}$ 이다. 위에서 나타난 과정은 앞서 구한 식 (3)에 모터 내 마찰계수를 빼고 Drag에 의한 저항토크로 적용한다. 그리고, 모터의 속도(각가속도)의 변화가 없어야 하므로 다음 식이 성립하게 된다.

$$\frac{dw}{dt} = 0 = K_T \frac{v_i - K_e w}{R} - Drag \quad (13)$$

$$v_i = K_e w_d + \frac{R}{K_T} Drag = K_e w_{hover} + \frac{R}{K_T} dw^2(t) \quad (14)$$

위의 입력 전압 식에서 시간에 따른 $w^2(t)$ 가 계속 업데이트되어야 하므로 시뮬링크 상에서 이를 loop를 통해 input으로 쓰게 된다. 아래는 위 식 (14)를 matlab function(Speed-Voltage Conversion)에 적용한 것이다.

```

kinematics_practice5_2024b > Speed-Voltage Conversion
1 function v_i = fcn(Motor_Speed, Drag_Coeff, Motor_Speed_Des)
2
3   rad2rpm = 60 / (2 * pi);
4   rpm2rad = 1 / rad2rpm;
5
6   Resistance = 0.1170;           %[ohm]
7   K_Torque = 1 / (rpm2rad * 960); %[N m/A]
8   K_Emf = K_Torque;             %[V s/rad]
9
10
11   v_i = K_Emf * Motor_Speed_Des + Resistance/K_Torque * Drag_Coeff * Motor_Speed^2;
12
13
14
15
16
17 end

```

Fig. 7. Conversion Matlab function code

아래는 hovering 속도를 설정값으로 했을 때, 이를 반영한 시간에 따른 F450의 모터 속도를 나타낸다.

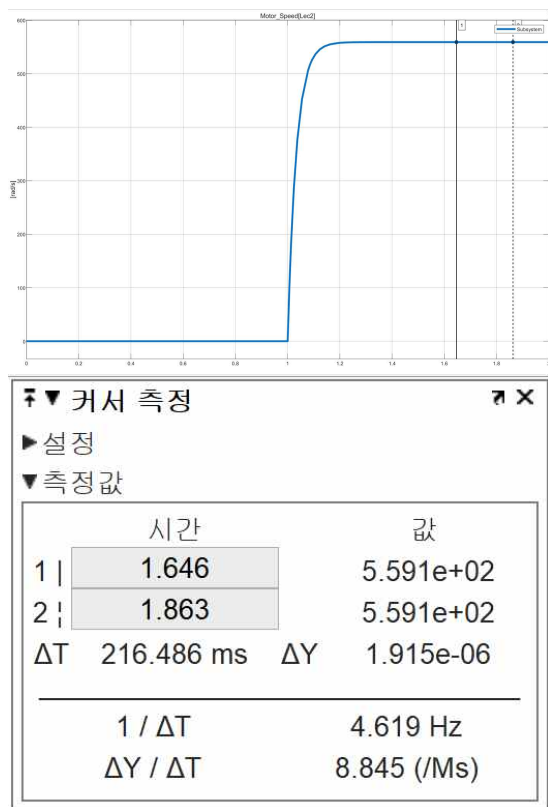


Fig. 8. Hovering F450 motor speed for t

호버링 속도는 step function으로 시간 지연 $\tau = 1$ 이 있는 상태로 들어가기에 1초부터 반응이 나타나게 된다. 시간에 어느 정도 지남에 따라, 모터의 속도가 559.1 rad/s에 가까워짐을 확인할 수 있다. 이는 요구 호버링 속도와 동일함을 알 수 있다.

다음으로 호버링 상태(또는 사용자 설정 모터 속도를 입력받을 때)에서 추력 T와 모멘트 F를 구하는 함수를 설계하였다. 지금까지는 단일 모터만 다루었기에 모터의 정확한 동특성을 기술하기가 어렵다. 따라서, 쿼드콥터의 모터 4개가 존재한 경우로 생각하여 이때 F450에 작용하는 추력과 모멘트를 각 모터마다 구하는 것이 목적이다. 앞으로는 위에서 만든 단일 Motor dynamic system을 그대로 두고 Mix라는 블록을 통해

4개의 모터 속도를 하나의 열벡터로 삼아 Input으로 넣게 된다. 그리고, 각각의 요구 모터 속도에 맞추어 시간에 따른 실제 모터 속도를 제작한 T,M Finder 함수에 넣어 추력 T와 모멘트 F를 확인하게 된다.

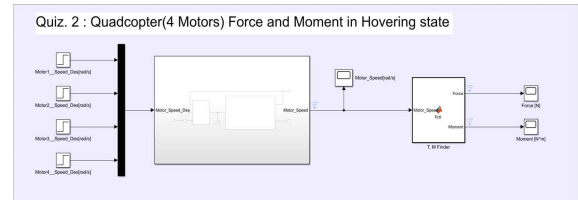


Fig. 9. F450 Thrust and Moment per motor

위 시스템에 T,M Finder 함수를 구성하기 위해 호버링 상태의 쿼드콥터를 살펴보아야 한다.

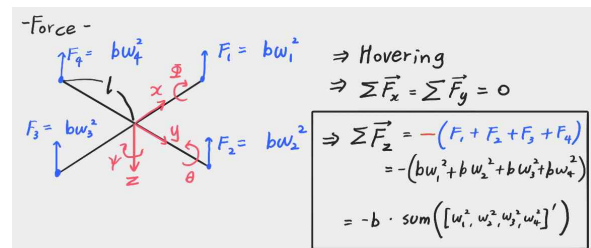


Fig. 10. Axis force per motor

각 축마다 모터에 의해 작용되는 힘을 보면 공중에 가만히 떠 있는 상태에서는 x, y 성분의 합력 성분은 존재하지 않는다. 오로지 z축에 의한 각 모터별 속도에 따른 추력만 존재한다. NED좌표계 기준, Z축에 대한 힘의 합은 위와 같다.

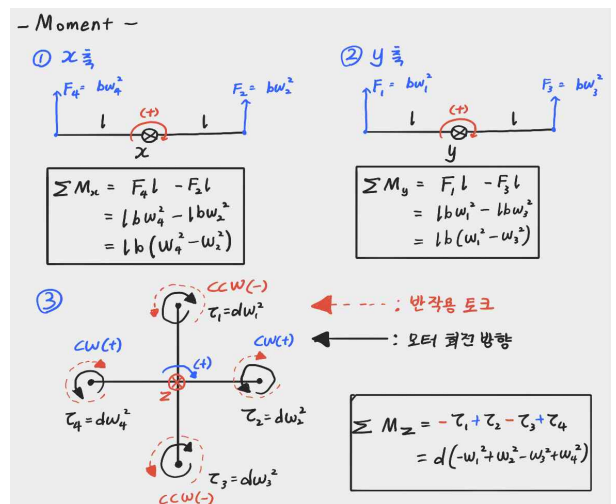


Fig. 11. Axis Moment per motor

x축과 y축 모멘트의 합은 그 축과 수직인 팔의 가장 자리에 위치한 모터 추력으로 결정되며, 이를 구하는 과정은 오른손 법칙에 따라 모멘트의 정의를 통해 위와 같이 계산된다. z축의 경우, 모터의 회전방향에 반대한 반작용 토크의 합이 쿼드콥터에게 작용하게 될 z

축에 대한 모멘트를 만들어낸다. 앞서 구성된 Motor dynamic subsystem에서 계산된 4x1 열벡터의 모터 속도들을 입력으로 받아 힘과 모멘트의 각 성분을 위에서 구한 식들로 T,F Finder에서 계산한다. 그 이후, 힘과 모멘트 각각 3x1 열벡터 형식으로 그 성분들을 저장해 scope로 확인한 결과가 아래와 같다.

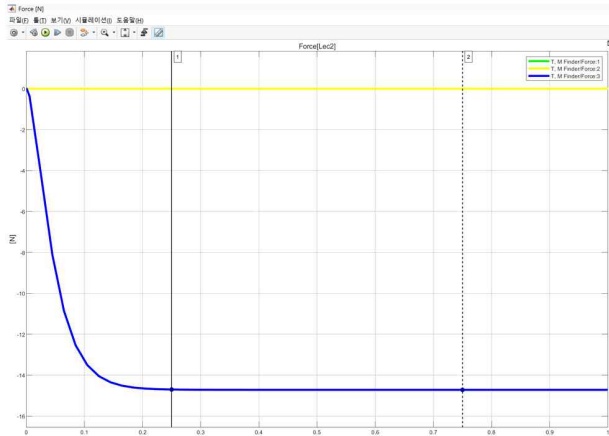


Fig. 12. Axis Force at hovering state

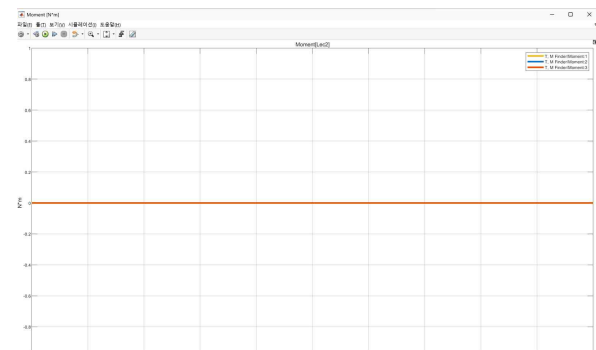


Fig. 13. Axis Moment at hovering state

여기서는 시간 지연 없이 호버링 속도를 설정하였으며, 그에 대한 결과로 z축의 Force만 증가하는 형태로 나타나고 나머지 x, y축의 Force와 3축 모두 Moment가 나타나지 않음을 확인할 수 있다. 여기서 Force는 NED 기준 z축이 아래를 향한다. 그렇기에 -z축으로 상승하는 상황이라 음수로 나타난 것이다. 또한, hovering 상태로 가기 전까지도 동일한 모터 속도로 자세 변환없이 상승하므로 모멘트가 모든 축에 대해 0인 것도 결과적으로 타당하다.

4) Finding maximum flight time at hovering

4주차에서는 Quadcopter의 모터 속도 변화를 주었을 때 어떻게 기동하는지에 대해 생각하는 시간을 가졌다. 앞서 Fig. 11.처럼 모터1~4에서 어떤 모터끼리 속도를 조절하면 상승 및 하강, 회전, 전진 및 후진을 하는지 확인했다. 각각의 경우마다 하나씩 간단히 그 예를 살펴보면 다음과 같다.

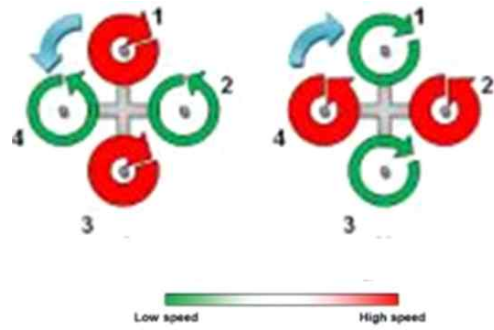


Fig. 14. Yaw Maneuvering

여기서 만약에 사진의 왼쪽처럼 1번과 3번의 모터 속도가 2, 4번의 모터 속도보다 증가하게 되면 1, 3번의 모터의 모멘트가 강해지게 되므로 이에 대한 반토크가 드론에 강하게 작용하여 반시계 방향으로 회전하게 된다.

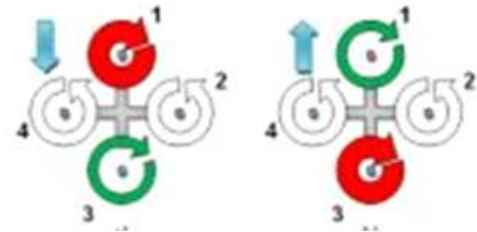


Fig. 15. Translation Maneuvering

다음으로는 오른쪽의 그림처럼 3번 모터의 속도가 1번 모터 속도보다 증가하게 되면 1번 모터보다 3번 모터에서의 추력이 강해지므로 초반에 추력 차이로 인해 -y축의 모멘트가 발생하여 드론이 x축 방향으로 기울어지게 된다. 잠시 시간이 지나 다시 힘의 평형과 모멘트 평형을 가지면서 기울어진 상태로 앞으로 전진하게 된다. 만약, 상승과 하강을 하고 싶다면 간단하게 모든 모터의 입력 전압을 높이거나 낮추면 된다.

만약에 호버링 상태의 F450의 최대 체공시간은 어떻게 될까? 이를 알기 위해서는 공기역학적 관점에서 자주 쓰이는 Momentum Theory를 적용해야 한다

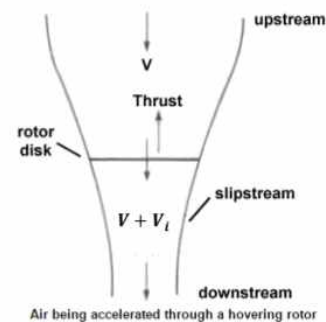


Fig. 16. Actuator disk theory

먼저 프롭을 매우 얇은 원판으로 간주한다. 이는 wingtip에 의한 회전 효과와 와류를 무시하게 해준다. 또한, 유체는 현재 마하급의 유속을 다루는 것이 아니므로 비압축성 및 비점성인 이상유체로 가정하며, steady state로 가정한다. 그럴 때, rotor disk를 통과하는 면적에 대해 균일한 유도속도 V_i 가 존재한다. 여기서 upstream인 유속은 V , 원판 단면을 지나가는 slipstream에서는 프롭을 지나가며 $V + v_i$ 로 변하게 된다. 그 아래로 더 내려가게 되면서 프롭을 빠져나오는 downstream에서는 유속이 $V + 2v_i$ 로 변화하게 된다. 비압축성 및 정상 상태에서의 질량 유량을 고려할 때, 프롭을 통과하는 질량 유량은 다음과 같다.

$$\dot{m} = \rho A (V + v_i), \quad A = \pi R^2 \quad (15)$$

추력 T 는 질량유량에 프롭의 Upstream과 Downstream 사이의 속도 변화의 곱으로 정의할 수 있다.

$$T = \dot{m} \Delta V = \rho A (V + v_i) \times 2v_i \quad (16)$$

호버링 상태에서는 $V = 0$ 이므로 T 는 아래 식과 같고, 이를 v_i 에 대해 나타내면 다음과 같다.

$$T = 2\rho A v_i, \quad v_i = \sqrt{\frac{T}{2\rho\pi R^2}} \quad (16)$$

또한, 이러한 추력을 만들어내기 위해 필요한 이상적인 동력 Power는 다음과 같다.

$$P = T(V + v_i), \quad P = \frac{T^{\frac{3}{2}}}{\sqrt{2\rho A}} \quad (16)$$

동력의 정의는 시간에 따른 에너지의 소모량과 같다. 이를 통해 다음 동력에 대한 채공 시간을 구하면 되겠지만, Figure of Merit을 적용해 현실에 가까운 동력으로 근사시킬 필요가 있다. 실제 power에 대한 이상적인 power의 비율은 1을 넘길 수 없다. 따라서, 이번 수업에서는 Figure of Merit = FM = 0.4정도로 설정하여 (16)에서 구한 동력 P 를 사용하였다.

Table 3. F450 Prop and battery parameters

Symbol	Value	Unit	Description
R	0.119	m	프로펠러 반지름
FM	0.4		Figure of Merit
U_batt	5,000	mAh	배터리 용량
V_batt	14.8	V	배터리 전압

위의 제시된 테이블의 파라미터를 가지고 matlab에서 연산을 진행한 결과는 아래와 같다.

```
C:\Users\user\Desktop\gnc_4th4.m
1 clear all
2 clc
3
4 %% Load parameters
5 m=1.762; % [kg]
6 rho=1.2; % [kg/m^3]
7 g=9.81; % [m/s^2]
8 R=0.119; % [m]
9
10
11 FoM = 0.8;
12 Cap = 1500; % [mAh]
13 VBat = 14.8; % [V]
14
15 %% Thrust of one propeller
16 Tprop = 1/4*m*g % [N]
17
18 %% Ideal power from momentum theory
19
20 Ap = R^2*pi; % propeller disc
21
22 % ideal power at hover
23 Pideal = Tprop^(3/2)/(sqrt(2*rho*Ap)) %[W]
24
25 %% real power
26 % Power at one propeller
27 Preal = Pideal/FoM
28 % Power for quadrotor
29 Pt = 4*Preal
30
31 %% Flight time
32 % Battery energy
33 Ebat = VBat*Cap/1000 % [Wh]
34 % Maximum flight time
35 time = Ebat/Pt % [h]
36 % time in minutes
37 timemin = time*60 % [min]
```

Fig. 17. Maximum flight time mat.

Tprop =	Pt =
4.3213	137.4564
Pideal =	Ebat =
27.4913	22.2000
Preal =	time =
34.3641	0.1615
Pt =	timemin =
137.4564	9.6903

Fig. 18. Result from mat. file

위의 결과로 F450의 최대 호버링 시간이 9.6903분임을 알 수 있다.

5) Modeling Body & Kinematic system(5주차)

지금까지 F450의 성능을 확인했다면, 5주차부터는 상황별 좌표계를 설정하여 F450의 위치 및 자세를 정의하게 된다. 앞서 추력과 모멘트를 계산할 때 NED좌표계를 설정하였다. 이는 즉 Body fixed frame으로서 사용한 동체좌표계이고, 우리는 절대적인 좌표계인 관성좌표계가 따로 필요하다. 그래야 드론의 위치 및 자세를 조종자 입장에서 파악이 가능하기 때문이다. 실제 세상은 절대적인 좌표계가 존재하지 않지만, 가정을 통해 지구관성좌표계인 Earth fixed frame으로 설정한다. 지구는 드론의 움직임에 비해 매우 느리게 자전 또는 공전하므로 지구는 정지한 상태라 가정한다.

이제 필요한 방식은 F450의 동체좌표계의 정보를 지구 고정 좌표계로 변환하는 것이다. 이는 다음과 같은 변환 행렬에 의해 변환하고자 하는 벡터에 적용하면 된다.

$$R_z(\psi) = \begin{bmatrix} c(\psi) & -s(\psi) & 0 \\ s(\psi) & c(\psi) & 0 \\ 0 & 0 & 1 \end{bmatrix}, R_y(\theta) = \begin{bmatrix} c(\theta) & 0 & s(\theta) \\ 0 & 1 & 0 \\ -s(\theta) & 0 & c(\theta) \end{bmatrix}, R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c(\phi) & -s(\phi) \\ 0 & s(\phi) & c(\phi) \end{bmatrix}$$

$$R_E^B = R_z \cdot R_y \cdot R_x = \begin{bmatrix} c(\psi)c(\theta) & c(\psi)s(\theta)s(\phi) - s(\psi)c(\phi) & c(\psi)s(\theta)c(\phi) + s(\psi)s(\phi) \\ s(\psi)c(\theta) & s(\psi)s(\theta)s(\phi) + c(\psi)c(\phi) & s(\psi)s(\theta)c(\phi) - c(\psi)s(\phi) \\ -s(\theta) & c(\theta)s(\phi) & c(\theta)c(\phi) \end{bmatrix}$$

Fig. 19. Rotation Matrix for Transformation

위와 같은 방식을 오일러 회전 행렬 방식이라 한다. 이를 사용할 때 주의할 점은 pitch의 경우 theta가 $-\pi/2 < \theta < \pi/2$ 이라는 짐벌락 조건이 있다는 것이다. 만약 theta가 -90° 혹은 90° 에 도달하게 되면 두 회전축이 일치하게 되어 자유도가 줄어들어 그 자세를 정확히 설명하기 어렵게 된다. 이러한 조건 설정은 위의 simulink 구성도에서 설명할 것이다.

또한, 회전행렬은 Orthonormal matrix이므로 그 행렬의 역행렬과 전치행렬의 형태가 같다는 특징을 보인다. 따라서, 직접 역행렬을 구하지 않고도 전치행렬을 구해 적용하는 편리함을 지닌다. 따라서, Body $\rightarrow$ Earth 또는 Earth $\rightarrow$ Body로의 좌표계 변환은 다음의 과정을 거쳐 변환 대상 벡터에 곱해주면 된다.

$$R_E^B = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$$

$$R_B^E = R_x^T(\phi) \cdot R_y^T(\theta) \cdot R_z^T(\psi)$$

$$R_B^E = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & \sin \phi \\ 0 & -\sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix} \begin{bmatrix} \cos \psi & \sin \psi & 0 \\ -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_B^E = \begin{bmatrix} c\phi c\psi & -s\phi c\psi & s\phi s\psi \\ s\phi c\psi & c\phi c\psi & c\phi s\psi \\ -s\psi & c\psi & 0 \end{bmatrix}$$

where $c\phi \triangleq \cos \phi$ and $s\phi \triangleq \sin \phi$.

Fig. 20. Body $\leftrightarrow$ Earth Coordinate change

Angular velocity 변환은 다음의 과정을 따라 변환행렬을 적용하면 아래의 최종 변환행렬을 얻게 된다.

$$\begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} \dot{\phi} \\ 0 \\ 0 \end{bmatrix} + R_x^T(\phi) \begin{bmatrix} 0 \\ \dot{\theta} \\ 0 \end{bmatrix} + R_x^T(\phi) \cdot R_y^T(\theta) \begin{bmatrix} 0 \\ 0 \\ \dot{\psi} \end{bmatrix}$$

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin \phi \tan \theta & \cos \phi \tan \theta \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi \sec \theta & \cos \phi \sec \theta \end{bmatrix} \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

Fig. 21. Angular velocity transformation

5주차에서는 이와 함께 중력에 의한 F450이 지구 고정 좌표계에서의 받는 힘을 구하는 과정도 진행하였다.

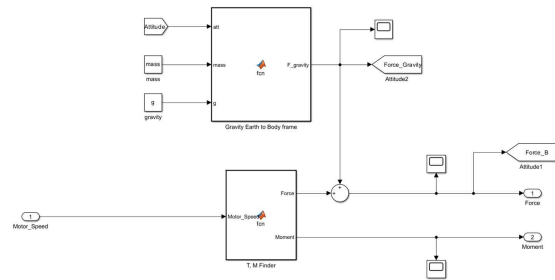


Fig. 22. Gravity Earth to Body frame

이 과정에서는 앞서 Fig. 10.에서 나타난 그림처럼 동체좌표계 기준 z축 방향으로 중력이 F450에 작용하고 있다. 하지만, 우리가 알 수 있는 사실은 실제 중력의 방향이 물체와 지구 중심 사이를 잇는 방향으로만 작용함을 알고 있다. 즉, 지구 고정 좌표계 기준으로

$$g_{earth} = [0 \ 0 \ g]^T \quad (16)$$

위와 같음을 알 수 있다. 하지만, F450의 동특성을 먼저 동체좌표계에서 해석하고 이를 변환행렬을 통해

알아야 하므로 동체좌표계 기준 g_{body} 를 알아야 한다. 이는 Earth $\rightarrow$ Body로의 좌표계 변환을 통해 구하면 된다. 여기서 input값으로 알아야 하는 것은 질량과 중력가속도도 중요하지만, 시간에 따라 드론의 자세이다. 추후 계산될 Attitude를 인풋으로 받아 아래와 같이 구성하게 된다.

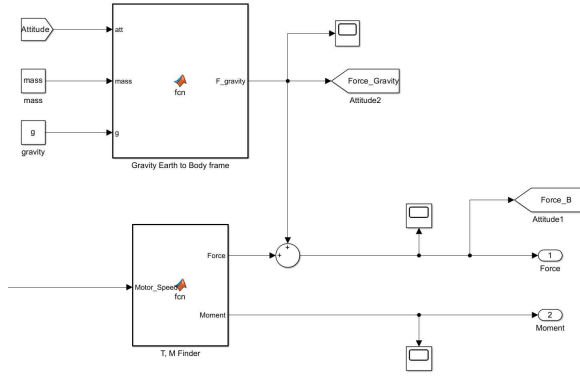


Fig. 23. Gravity Earth to Body frame slx.

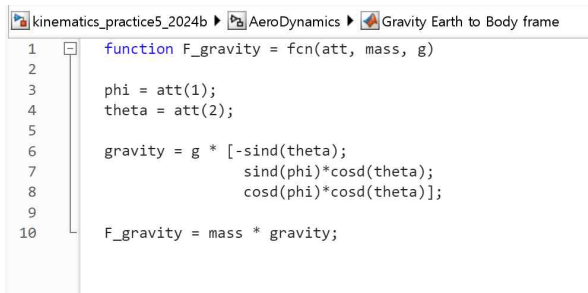


Fig. 24. Gravity Earth to Body frame func.

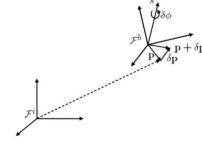
T,F Finder Matlab function 블록 위에 위치한 Gravity Earth to Body frame Matlab function 블록 내에 적용된 gravity 코드를 확인해보자. 앞서 지구 고정 좌표계의 중력가속도 x, y 성분이 0이므로 변환행렬의 1열과 2열과 연산하지 않게 되었기에 위와 같이 3열에 대해서만 간단하게 나타낼 수 있다. 이렇게 구한 동체좌표계에서의 중력은 모터의 성능에 의해 발생하는 힘과 sum블록으로 더해져 F450의 위치와 자세, 속도 등을 계산하는 과정으로 넘어가게 된다.

6) Final Body & Kinematic system(7주차)

7주차에서는 앞서 Input으로 받는 F450의 동체좌표계 기준 받는 힘과 모멘트를 설정하고 Rigid Body Dynamics 이론 내 Transport Theorem을 적용해 F450의 위치, 자세, 속도, 각속도를 구하는 것이 최종 목표이다. 이는 관성 좌표계와 회전 좌표계 사이에 회전 좌표계에 운동하는 물체를 관성 좌표계에서 기술할 수 있는 매우 유용한 이론이다.

6-DoF EOM(Equation Of Motion)

Rigid Body Dynamics



Rigid body can be completely described by the velocity at the CoG and the rotational velocity

$$\frac{d}{dt} \mathbf{p} = \frac{d}{dt} \mathbf{p} + \boldsymbol{\omega}_{B/I} \times \mathbf{p}$$

Transport Theorem

Fig. 25. Rigid Body Dynamics

이 이론은 어떤 물체의 형상과 질량 분포에 관계없이 그저 질량중심에 질점을 두고 해석하는 것이다. 그렇기에 이후에 나오는 식들 중 2차 관성 모멘트를 필요로 하는 경우가 발생한다. 이러한 과정을 요약하면 다음과 같이 나타낼 수 있다.

$$\begin{pmatrix} \dot{p}_n \\ \dot{p}_e \\ \dot{h} \end{pmatrix} = \begin{pmatrix} c\theta c\psi & s\phi s\theta c\psi - c\phi s\psi & c\phi s\theta c\psi + s\phi s\psi \\ c\theta s\psi & s\phi s\theta s\psi + c\phi c\psi & c\phi s\theta s\psi - s\phi c\psi \\ s\theta & -s\phi c\theta & -c\phi c\theta \end{pmatrix} \begin{pmatrix} u \\ v \\ w \end{pmatrix}$$

$$\begin{pmatrix} \dot{u} \\ \dot{v} \\ \dot{w} \end{pmatrix} = \begin{pmatrix} rv - qw \\ pw - ru \\ qu - pv \end{pmatrix} + \frac{1}{m} \begin{pmatrix} f_x \\ f_y \\ f_z \end{pmatrix},$$

$$\begin{pmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{pmatrix} = \begin{pmatrix} 1 & \sin\phi \tan\theta & \cos\phi \tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \frac{\sin\phi}{\cos\theta} & \frac{\cos\phi}{\cos\theta} \end{pmatrix} \begin{pmatrix} p \\ q \\ r \end{pmatrix}$$

$$\begin{pmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{pmatrix} = \begin{pmatrix} \frac{J_y - J_z}{J_x} qr \\ \frac{J_z - J_x}{J_y} pr \\ \frac{J_x - J_y}{J_z} pq \end{pmatrix} + \begin{pmatrix} \frac{1}{J_x} \tau_\phi \\ \frac{1}{J_y} \tau_\theta \\ \frac{1}{J_z} \tau_\psi \end{pmatrix}.$$

Fig. 26. 6-DOF Equation of Motion

첫 번째는 NED 좌표계 기준에서의 위치 변화와 body 속도와와의 관계를 나타내고, 두 번째 식은 $rv - qw$ 와 같은 관성 교차항(코리올리 성분)과 body에 작용하는 외력(중력)이 합쳐져 나타나는 선속도의 변화량 관계식을 의미한다. 세 번째 식은 각속도를 변환하여 지구 고정 좌표계 기준에서의 각속도로 변환해준다. 네 번째는 관성모멘트 불균형에 의한 자이로적 coupling인 첫 항과 외부 토크에 관성모멘트를 나눈 두 번째 항을 더해 각가속도를 찾는 역할을 한다.

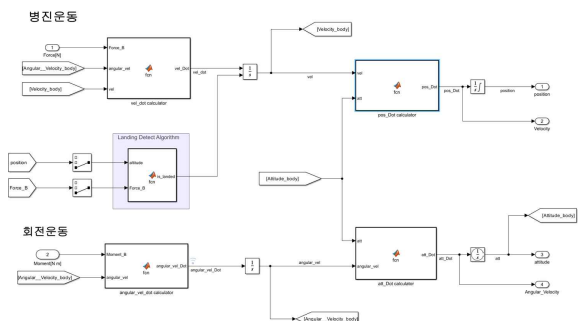


Fig. 27. Body & Kinematic system

위의 시스템 구조는 5주차와 7주차를 거쳐 설계하게 된다. 먼저, vel_dot calculator에는 Body_Force, Body_Moment에 대해 transport Theorem을 적용한 식을 코드로 설정했다. 외적된 부분은 matlab 내장함수인 cross를 통해 간단히 계산할 수 있다.

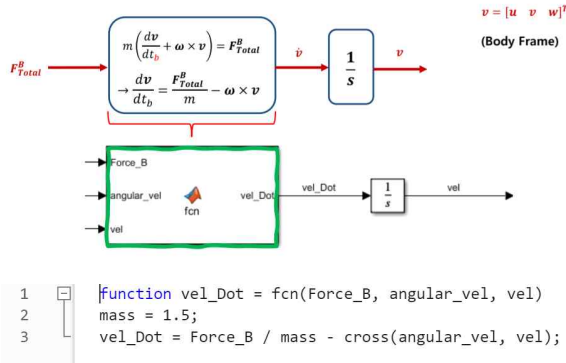


Fig. 28. vel_dot calculator code

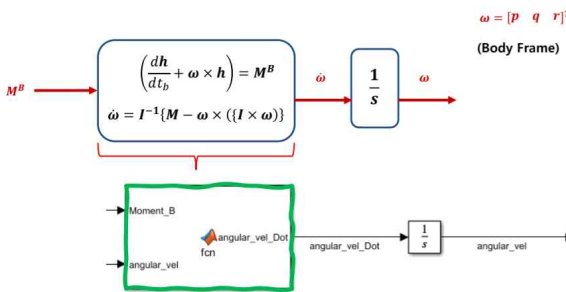


Fig. 29. angular_vel_dot calculator code

위 과정을 통해 구해진 Body frame 상에서의 속도와 각속도를 5주차에서 배운 변환행렬을 적용해서 지구 고정 좌표계 상에서의 속도와 각속도로 변환한다. 그 부분은 아래와 같다.

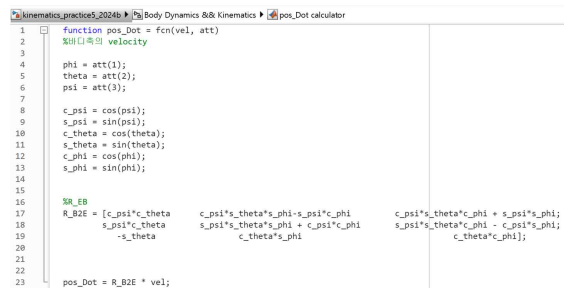


Fig. 30. pos_dot calculator code

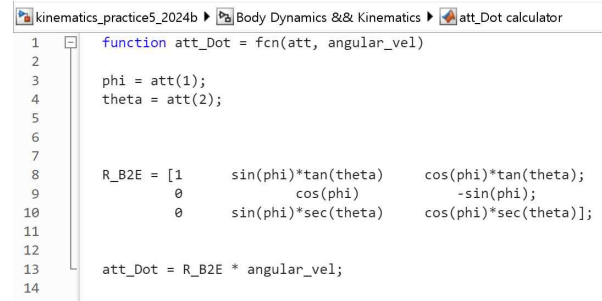


Fig. 31. vel_Dot calculator code

하지만, 지면 기준을 설정하지 않고 이대로 적용한다면 힘이 지면으로 들어가는 방향으로 바뀔 시 지면을 뚫고 들어가는 위치가 나타나는 현상이 생길 수 있다. 따라서 Landing Detection Algorithm이 필요하게 된다. 이를 위해서 vel_Dot을 적분하는 과정에서 적분기에 외부 재설정을 '레벨 유지' 상태로 변경하고 이를 판별해줄 알고리즘에 의한 logical값을 받게 해야한다.

조건으로는 altitude인 $z = \text{position}(3) \geq 0$ 이어야 하며 동시에 동체좌표계인 NED 기준 $\text{Force}_z = \text{Force}_B(3) > 0$ 인 경우가 되면 착륙했음을 의미하는 True값을, 그 두 가지 조건 중 하나라도 만족하지 않는다면 False값을 적분기에 보내 지면 상일 경우 position을 업데이트하지 않도록 하는 것이다. 아래의 그림처럼 이러한 z축 성분은 selector로 분류하여 input에 적용하였고, 알고리즘 내 코드는 교안의 두 조건문보다 효율적인 한 줄로 작성하였다.

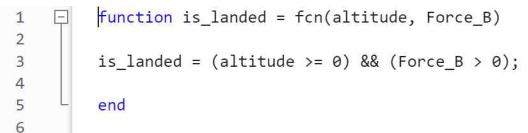
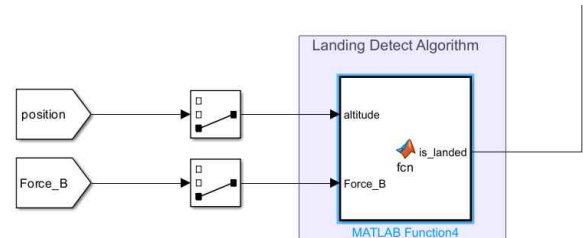


Fig. 31. Landing Detect Algorithm code

최종적으로 세 가지 Dynamic systems를 결합한 형태는 아래와 같다. 그 위에 보이는 것은 결과로 나온 지구 고정 좌표계 상에서의 위치, 선속도, 자세, 각속도를 scope로 바로 확인할 수 있도록 배치해 두었다.

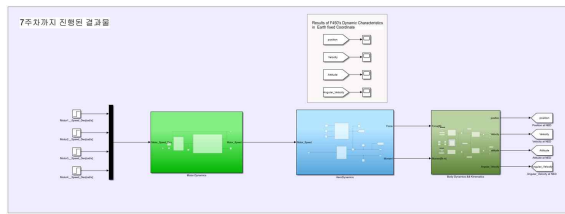


Fig. 32. Total F450 Dynamic system

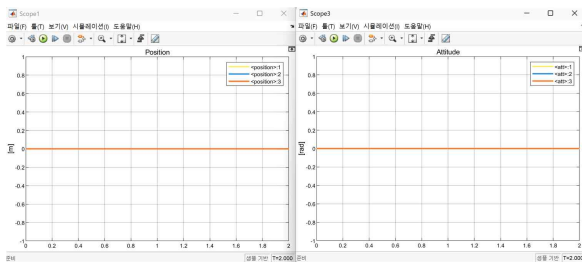


Fig. 33. Position & Velocity at Hovering

앞서 구했던 Hovering speed = 559.11rad/s일 때 F450의 위치와 자세를 보면 전혀 변화하지 않음을 확인할 수 있다. 당연히 Hovering 상태에서는 고정된 위치와 자세를 가지므로 결과가 타당함을 알 수 있다.

그렇다면 모터의 속도를 다르게 설정한다면 어떤 결과가 나오는지 확인하면 다음과 같다. 먼저, 1번 모터의 속도를 20rad/s 더 증가시키고 3번 모터를 20rad/s 더 감소시킨다. 이렇게 되면 조금 기울어진 상태로 NED 기준 -x축 방향을 따라 후진을 하게 될 것이라 예측할 수 있다.

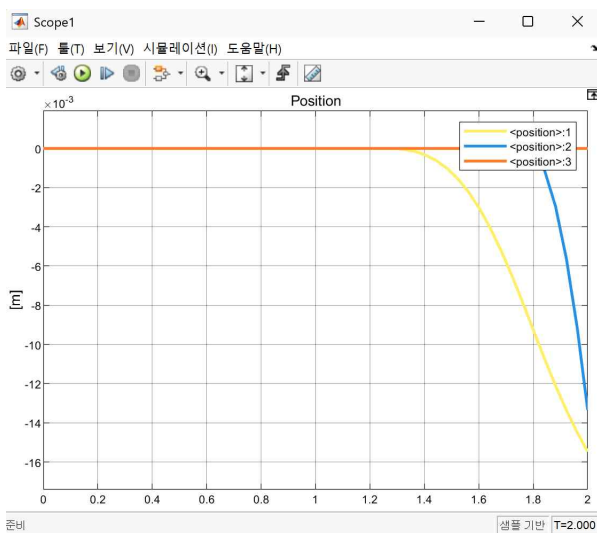


Fig. 34. Position at backward state

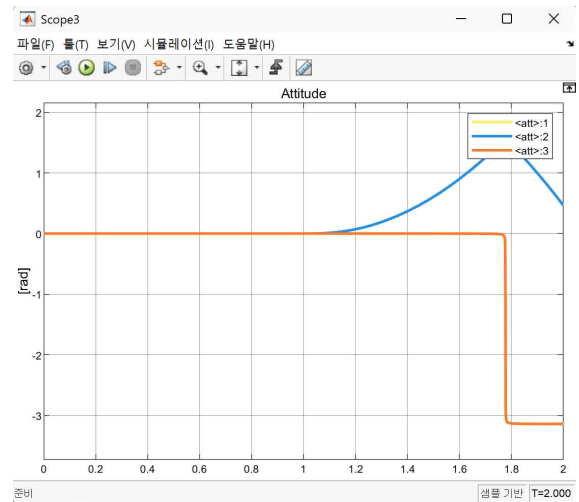


Fig. 35. Velocity at backward state

결과를 확인해보면 position의 경우 z변화는 일어나지 않는 것이 맞고 노란색의 x축 변화와 파란색의 y축 변화가 -방향으로 가고 있음을 확인할 수 있다. 그 말은 NED 좌표계 기준 제 3사분면을 향해서 기울어져 가고 있는 것이라 분석되는데 이는 예측과 다르게 된다. 그렇다면 Attitude를 봤을 때 확인되는 점도 갑작스런 변화가 t = 1.7s 부근에서 발생한다. 이에 대한 이유를 확인해보고자 angular_velocity의 그래프를 확인해보니 다음과 같이 그 지점에서 발산하는 형태로 나타났다.

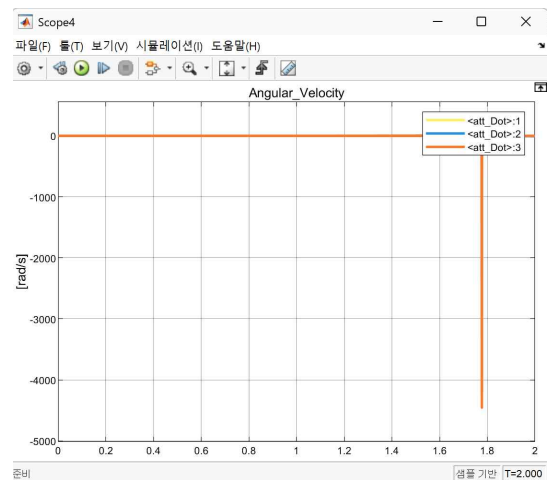


Fig. 36. angular vel at backward state

이러한 현상이 발생하는 이유를 생각했을 때 앞서 angular_velocity를 NED -> Earth로 변환하는 행렬에 대해 짐벌락 현상이 있다고 언급했다. 현재 준 속도 변화에 의해 드론은 현재 구현된 시뮬링크 상에서는 계속 회전하게 되므로 그 짐벌락 구간에 도달했을 때 이렇게 발산되어 오류가 난 것이라 생각된다. 이를 해결하기 위해 쿼터니언 기법을 적용하는 방법이 있지만, 현재 수업 내용 과정 내로 제작하는 방향이 맞다고 보아 여기까지 분석을 마치는 것이 좋다고 생각한

다.

후 기

지금까지 그저 이론으로만 접해왔던 비행동역학 및 자동 제어 시스템의 구성을 Simulink 상으로 직접 구현해보며 이에 대한 결과를 즉각적으로 확인하는 좋은 기회를 가지고 있음에 감사하다. 앞으로도 구현될 더 많은 부분들을 통해 실제 F450과 거의 동일한 Dynamic system을 설계하길 기대한다.

참고문헌

1. , 유도제어시스템 교안(1-7주차)
2. 위키피디아